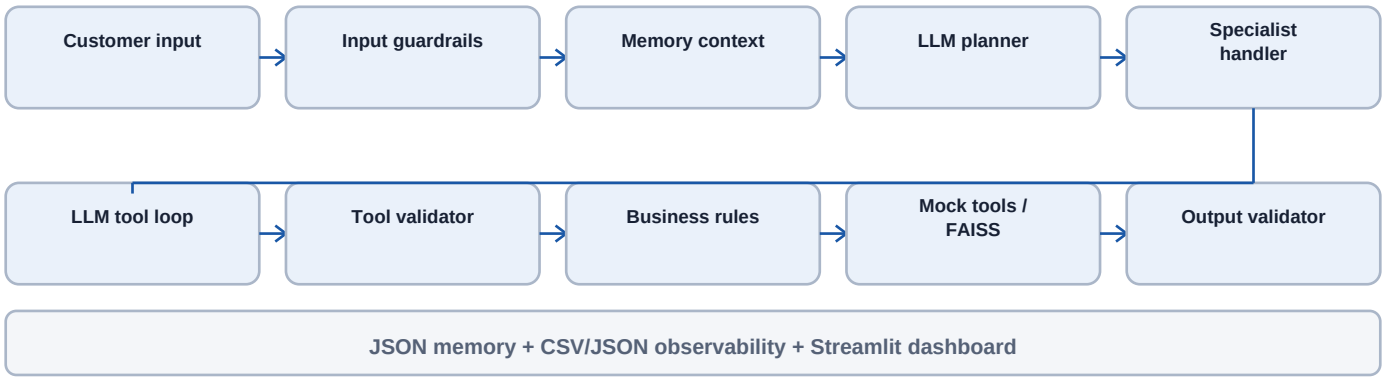
Customer Operations Agent

Tier 1 + Tier 2 architecture, controls, and engineering tradeoffs

Architecture



Key design decisions and tradeoffs

- One graph-based agent with an LLM router and focused specialists: explicit control flow and simpler recovery, at the cost of two model stages.
- The model chooses tools, but deterministic Pydantic validation and the BusinessRuleEngine decide whether actions execute.
- FAISS and JSON keep the assignment self-contained and inexpensive; they are not multi-process or horizontally scalable stores.
- Mock business systems make runs safe and repeatable while preserving realistic budgets, approval gates, tickets, and audit traces.
- Policy answers fail closed without KB citations; this reduces hallucination risk but can increase safe escalations when retrieval fails.

Guardrail approach

- Before the LLM: prompt-injection detection, PII masking, abuse review, and blocked-input routing.
- Before each action: tool allowlist, strict arguments, customer isolation, maximum calls, dead-loop detection, and circuit breaking.
- Financial controls: refunds above INR 1,000 require human handling; offers are at most 20% with a total INR 10,000 run budget.
- Before delivery: structured output schema, confidence threshold, citation requirement, PII masking, and safe fallback.

Evidence, rollout, cost, and risk

Submission scope: Tier 1 and Tier 2; Tier 3 is not claimed

Verification evidence

- 42 deterministic tests pass, including 12 integration scenarios for billing, refund, cancellation, technical support, injection, abuse, ambiguity, budgets, approval, circuit breaking, and loops.
- Scripted model tests prove genuine tool-selection flow: customer lookup, KB search, grounded citation capture, cross-customer rejection, and large-refund escalation.
- Offline CLI and Streamlit dashboard render successfully; every graph request and tool call is logged.
- Live OpenAI connectivity reached the provider but returned 429 insufficient_quota. The graph logged the failure and returned a safe human-support response.

Production rollout plan

- 1. Replace local mocks with sandbox adapters while retaining the same validated tool contracts.
- 2. Run in shadow mode: compare proposed actions with human decisions and block all writes.
- 3. Canary low-risk billing and troubleshooting requests; keep refunds and offers approval-only.
- 4. Add durable conversation, approval, and budget storage with idempotency keys and role-based access.
- 5. Add distributed tracing, alerting, secrets management, model/version pinning, and retrieval-quality monitoring.
- 6. Expand gradually only when safety, groundedness, latency, cost, and business KPI thresholds remain healthy.

Cost and risk notes

- Primary variable costs are planner/specialist tokens and KB embeddings; cache stable embeddings and keep prompts/tool results compact.
- Retention discounts directly reduce revenue, so eligibility and aggregate budget are enforced in code rather than prompts.
- Provider quota, latency, and outages can interrupt automation; retries, circuit breakers, observability, and human fallback limit impact.
- PII and payment data create privacy risk; mask before model use, minimize logs, encrypt durable stores, and apply retention policies.
- Model confidence is self-reported and not calibrated; production thresholds require empirical validation against human-reviewed outcomes.

Business KPIs

- Resolution rate and escalation rate by intent.
- Net retained monthly revenue minus discount cost.
- Cost per resolved request and p95 end-to-end latency.
- Grounding failures, approval violations, injection blocks, and incorrect tool calls.

Current recommendation: demonstrate with a funded API key, then record the required 2-4 minute live video.